CONCEPTS OF OPERATING SYSTEM

Made By : **KASHIF SAYYAD**

MODULE I — INTRODUCTION TO SOFTWARE AND OPERATING SYSTEM

■ Introduction to Software and Classification

Software is a set or collection of programs, procedures and associated documentation which instructs the computer hardware what to do and how to do it. In short, software is the soul of the computer; without software the hardware is only a dead machine.

Classification of software:

- **System software** — software that controls and co-ordinates the computer hardware and provides a base on which the application programs run: operating systems, assemblers, compilers, interpreters, device drivers and linkers.
- **Application software** — programs written to satisfy the need of the end user: word processor, spreadsheet, DBMS software, antivirus and games. It may be general-purpose (MS-Office) or application-specific (payroll, railway reservation).
- **Utility software** — helper tools that maintain the disk, files and system security: disk formatting, defragmenter, backup, file compression.

The **Operating System** is the main and most important component of system software. It is defined as a set of programs that manages all the resources of the computer (CPU, memory, devices, files) and works as an interface between the user and the hardware. Running **programs and processes**: a program is a passive set of instructions stored on disk, whereas a process is that program in actual execution in memory.

■ Operating System Fundamentals — Structure, Types and Functions

Structure of Operating System: from inside to outside the layers are — hardware, **kernel** (the core which directly manages CPU, memory and devices), **system programs / system calls**, **shell or command interpreter** (accepts commands of the user) and the **application programs / users**.

Types of Operating System: Single user OS, where only one user can work at a time and the entire machine resource is given to him (MS-DOS is the classic single user, single tasking system); and **Multi user OS**, where many users are connected through terminals and share the same computer simultaneously (UNIX and Linux are multi user systems).

Basic functions of an Operating System are:

1. **Process management** — creating and scheduling of processes, and synchronisation and communication between them.

2. **Memory management** — keeping track of memory, allocating and deallocating memory to processes.
3. **File management** — creating and deleting files and directories, and controlling their access.
4. **Device (I/O) management** — controlling input-output devices with the help of device drivers.
5. **Security and protection** — protecting data and resources of each user against unauthorised access.
6. **Command interpretation** — taking the commands of the user and getting them executed.

Characteristics of a modern operating system: today's OS is **multi-programming, multi-tasking, multi-user, multi-processing, multi-threading, time-sharing** and uses **virtual memory**; it supports graphical user interfaces, networking, long file names, plug and play devices and virtualisation.

■ Evolution of Operating System

The operating system did not appear at a moment; it developed step by step with the development of the hardware:

1. **Early systems** — no operating system; the programmer had to sign up for machine time and manually load the program from cards or paper tape.
2. **Simple batch systems** — jobs with similar needs were grouped together and processed one after another by an operator; controls cards and monitor programs came into use.
3. **Multiprogramming batch systems** — several jobs are kept in memory at the same time; when the running job waits for I/O, the CPU is switched to another job, which increases CPU utilisation.
4. **Time sharing (multitasking) systems** — the CPU time is divided into small slots and rotated among the jobs of several users, so every user feels that the machine is his own.
5. **Operating system for personal computers, workstations and hand-held devices** — simple user-friendly OS (DOS, Windows, iOS, Android) made for a single desktop or mobile device.
6. **Parallel systems** — many processors are connected to one shared memory and work simultaneously on the same job for very high speed.
7. **Distributed systems** — several independent computers connected through a network appear to the user as one single system; work, files and resources are shared.
8. **Real-time systems** — these give their output strictly within a fixed time limit, and are therefore used in process control, missiles, aircraft and medical instruments.

Advantages and disadvantages of each system (as required by the syllabus): batch processing gives efficient use of the machine but has no user interaction and long turnaround time; multiprogramming increases throughput but demands careful memory management; time sharing gives quick response but the design of the scheduler is complex; distributed systems give resource sharing, speed and reliability but need network security; real-time systems give guaranteed behaviour but need special-purpose and costly hardware and software.

MODULE II — PROCESS MANAGEMENT AND CPU SCHEDULING

■ Concept of Process, Process States, Process Control Block

A **process** is a program in execution — it is an active entity which has a program counter, registers and a current state, while a program is only a passive file of stored instructions. **Process states** in a simple model are: **New** (process is being created), **Ready** (loaded in memory, waiting for the CPU), **Running** (instructions are being executed), **Waiting / Blocked** (waiting for some event like I/O completion), and **Terminated** (finished).

The operating system keeps all management information of every process in a data structure in main memory called the **Process Control Block (PCB)**. It contains: **process identity number** and pointers, **process state**, **program counter** and the contents of **CPU registers** (so the process can be stopped and restarted), **CPU-scheduling information** (priority, pointers to queues), **memory management details**, and **accounting information** (CPU time used, time limits) and **list of open files and devices**.

Operations on processes are process creation (by fork in UNIX), process termination and the changing of process state from one to another. A **thread** is the smallest unit of execution inside a process; a process can have many threads which share the same memory and resources of the process but keep their own program counter and registers.

■ CPU Scheduling — Types of Schedulers and Criteria

CPU scheduling is the basis of multiprogramming: it decides which of the ready processes will get the CPU next.

- **Long term scheduler (job scheduler)** — selects processes from the job pool on disk and loads them into main memory; it controls the degree of multiprogramming.
- **Short term scheduler (CPU scheduler)**— very fast; it selects one process out of all ready processes and gives the CPU to it. In the dispatcher the schedulers work.
- **Medium term scheduler** — swaps processes out of memory and back in to reduce the degree of multiprogramming.

Criteria for scheduling used to compare algorithms are: maximum **CPU utilisation** and **throughput** (number of processes completed per unit time) on one side and minimum **turn-around time** (submission to completion), **waiting time** (time spent in the ready queue) and **response time** (time until first response) on the other. Fairness of every user must also be kept.

■ Scheduling Algorithms — FCFS, SJF, Pre-emptive, Priority and Round Robin

1. First Come First Served (FCFS) scheduling algorithm — a non pre-emptive algorithm in which the process which comes first in the ready queue is served first; the implementation is a simple FIFO queue. It is easy and starvation-free, but short processes wait behind long processes; this is the **convoy effect**.

2. Shortest Job First (SJF) — the process with the smallest next CPU burst is executed first. It gives the minimum average waiting time, but the exact length of the next burst is not known in advance and is only estimated; long processes may get starvation. Its pre-emptive version is called **SRTF (Shortest Remaining Time First)**.

3. Pre-emptive scheduling — the running process can be interrupted at any moment by the scheduler (on the arrival of a better job or the expiry of a time slice) and the CPU is given to some other process. Non pre-emptive

scheduling, by comparison, lets the current process run until it finishes or terminates itself.

4. Priority Scheduling — every process is given a priority number and the CPU goes to the highest priority process first. Its main problem is **starvation (indefinite blocking)** of low-priority processes, solved by the technique of **aging** (slowly increasing the priority of waiting processes).

5. Round Robin (RR) — designed specially for time sharing; each process receives a fixed small time unit called a **time quantum** (say 10–100 ms). If the process does not complete within the quantum it is pre-empted and added back at the tail of the queue. It gives a good response time to every user and no starvation; too small a quantum gives excessive context switching, and too large a quantum turns it into FCFS.

MODULE III — INTER PROCESS COMMUNICATION, SYNCHRONISATION AND DEADLOCKS

■ Concurrent and Dependent Processes

Processes executing at the same time are **concurrent processes**; they may be independent (no relation, no shared data) or **dependent (co-operating) processes**, which can affect each other or are affected by other processes, produce or consume data and sometimes have to wait for each other's results. Synchronisation is needed for dependent processes so that a common data area is used in a consistent order; otherwise the value of the shared data may become a **race condition**; wrong result depending on the order of execution.

■ Critical Section and Semaphores

That part of a program where shared memory or shared files are used is called the **critical section**. Only one process must be inside its critical section at a time; this rule is **mutual exclusion**. A good solution must also satisfy **progress** (no unwanted blocking) and **bounded waiting** (fixed limit of waiting).

A **semaphore** is an integer variable on which only two atomic operations — **wait()** (down, P) and **signal()** (up, V) — are allowed. A wait is performed to enter the critical section and a signal to leave it. If the value of a semaphore is a single 0/1 it is a **binary semaphore (mutex)**, otherwise a **counting semaphore** which manages several identical resources. With semaphores the classic **producer–consumer problem** (producer puts items into a buffer, consumer removes them) is solved by one semaphore for the mutual exclusion of the buffer and two counting for the empty-full counting. **Methods of inter-process communication**: (a) **shared memory** — both processes agree on an area of common memory, and (b) **message passing** — *send* and *receive* with direct or indirect **mailboxes**; **process synchronisation problem** is to arrange their execution in a fixed order.

■ Deadlock — Concept, Modelling and Handling Methods

When a set of processes is blocked, each process holding some resource and waiting for another resource which is held by some other process in the same set, then that situation is a **deadlock**. Example: process P1 holds printer and needs tape, process P2 holds tape and needs printer — both wait forever. The four **necessary conditions** that must hold simultaneously are: **mutual exclusion** (only one process per resource), **hold and wait**, **no pre-emption**, and **circular wait**.

Deadlock modelling is done with a **resource allocation graph** whose vertices are processes and resources and whose directed edges are requests and assignments; if the graph has no cycle there is no deadlock, and a cycle in single-instance resources means deadlock.

Methods for handling deadlock:

1. **Deadlock prevention** — design the protocols so that at least one necessary condition can never hold, e.g. request all resources at the beginning, or strict ordering of resource requests.
2. **Deadlock avoidance** — keep checking each and every request before allocation, so that the system is **always in a safe state** and never enters an unsafe state; the **Banker's algorithm** is the standard avoidance algorithm using maximum need, allocation and available vectors.
3. **Deadlock detection and recovery** — allow deadlock to happen, find a cycle in a wait-for graph and recover by killing or rolling back victim processes, or by pre-empting resources.
4. **Deadlock ignorance (Ostrich algorithm)** — ignore it completely, used in most general operating systems like Windows and Linux.

Remember : Memory management makes the deadlock handling of buffer memory and page memory: the number of free buffers and physical frames of each process must be watched carefully so that deadlock through memory does not arise. Deadlock memory management in the syllabus refers to such careful allocation and avoidable re-ordering of buffers and memory requests.
